

FinPulse Project Report

SoFI AlgoLabs Core Induction, Assignment 1. Armaann Bhansali. GitHub: github.com/armaannbhansali/finpulse

Overview

FinPulse tracks 20 NSE listed Indian companies. It pulls live market data, stores it in a database, and shows it through a REST API and a dashboard. The two are deployed separately. Dashboard: finpulse-27phmb7xrifs4vg9xfnrge.streamlit.app. API: finpulse-api-yjel.onrender.com, with docs at /docs.

Architecture

There are three parts. algo1.py fetches data for the 20 tickers using yfinance and writes it into a SQLite database called finpulse.db. It uses an insert that updates existing rows instead of creating duplicates each time it runs. api.py is a FastAPI app that reads from the same database and serves it over REST, deployed on Render. dashboard.py is a Streamlit app that also reads from the database directly and turns it into tables and charts, deployed on Streamlit Community Cloud. The API and dashboard use separate requirements files, since the API has no need for Streamlit or Plotly at all.

Layer	Technology
Data source	yfinance
Database	SQLite
Backend	FastAPI, Uvicorn
Frontend	Streamlit, Plotly
Hosting	Render for the API, Streamlit Cloud for the dashboard

APIs used

GET /stocks returns all 20 companies with price, market cap, P/E ratio and EPS. GET /stocks/ticker returns one company, for example /stocks/RELIANCE.NS, and returns a 404 if the ticker isn't tracked. GET /market-summary returns total stocks tracked, average P/E and the largest market cap, calculated with SQL. FastAPI also generates interactive docs automatically at /docs.

Database design

One SQLite table called stocks, with columns for ticker, company name, price, market cap, P/E ratio, EPS and when it was last fetched. Ticker is the primary key, which is what lets the fetch script update existing rows instead of the table growing with duplicates every time it runs.

Features implemented

For the MVP: live and stored data for 20 companies across banking, IT, FMCG, auto, pharma and cement; SQLite storage that updates instead of duplicating; three REST endpoints; a dashboard with a sortable table, a historical price chart with adjustable time range, a comparison chart across companies using percentage change, and a bar chart comparing fundamentals.

Beyond the MVP: a toggle between line and candlestick charts using OHLC data that yfinance already returns, a refresh button on the dashboard that reuses the same fetch functions from algo1.py instead of duplicating them, and the API and dashboard being deployed as fully separate services.

Challenges faced

Two things were about equally hard here. One was the environment and deployment side, since this was my first time setting up a virtual environment, using Git properly, and deploying anything online. The other was actually understanding the concepts underneath, REST APIs, SQL, how a database update should work. A specific problem came up during deployment. My first requirements.txt was generated using pip freeze, which pulled in my entire local environment instead of just what the project needed. That caused the Render build to fail while trying to compile pyarrow, which turned out to be a dependency of Streamlit and Plotly that the API never actually uses. I fixed it by splitting the dependencies into a separate file just for the API. It also just made more sense once I thought about it, since the API build has no reason to install packages the dashboard needs.

Future improvements

Store historical prices in the database instead of calling yfinance live every time someone picks a date range, which would also let the API serve historical data directly. Add scheduled automatic refreshes instead of a manual button. Add login and personal watchlists. Add more fundamentals beyond P/E and EPS, like debt to equity or ROE.

AI usage disclosure

Used Claude to explain concepts, debug errors, and help structure this report.